

04 · Aktorik & Sicherheit

Mikrocomputertechnik 3 · Von Hardware-Countern, PWM-Signalen und dem rettenden Watchdog

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 69

Willkommen zu Block 4!

- Willkommen zur vierten Vorlesung in Mikrocomputertechnik 3
- Rückblick: In Block 3 hat der CLINT dem SoC einen Herzschlag (System-Tick) gegeben
- Bisher: CPU reagiert auf Ereignisse (Interrupts) und verarbeitet Daten
- Heute: das System wird aktiv — physikalische Welt steuern (Aktorik)
- Sicherheitsnetz: Wer passt auf die CPU auf, wenn Software hängt?

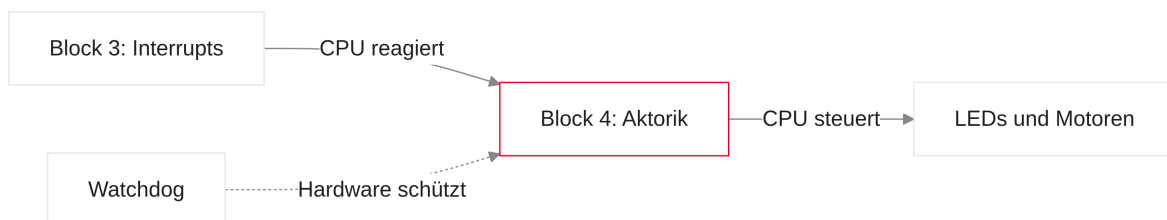
Notizen

Herzlich willkommen zurück! In Block 3 haben wir die Zeit gebändigt: System-Tick, CLINT und PLIC. Ein Mikrocontroller, der nur in sich hineinhört, reicht aber nicht. Heute steuern wir Ausgänge — Motoren, LEDs — und klären, wie Hardware bei Software-Abstürzen eingreift (Watchdog).

Folie

2 / 69

Einordnung im Semester



Notizen

Block 4 verbindet Ereignissteuerung mit Ausgabe und Ausfallsicherheit.

Anknüpfung: CLINT, Tick und volatile

Kurz-Recap aus Block 3 (Vertiefung im Übungsblatt):

- System-Tick: CLINT-Timer periodisch (typisch 1 ms)
- Foreground-ISR und Background-main() teilen die CPU-Zeit
- Geteilte Variablen: `volatile` gegen Compiler-Optimierung
- PLIC: Prioritäten und Ordnung bei externen Ereignissen
- Dieser Takt ist Grundlage für die Signalerzeugung heute

```
__attribute__((interrupt))  
void timer_isr(void) {  
    uptime_ms++;  
    write_mtimecmp(read_mtime() + TICKS_PER_MS);  
}
```

Notizen

Das Muster „Timer misst Zeit, Hauptschleife arbeitet“ ist die Blaupause vieler Embedded-Systeme. Heute nutzen wir präzise Zeitbasis und Peripherie-Timer, um Ausgangspins rhythmisch zu schalten — ohne die CPU zu blockieren.

Von der Zeitmessung zur Weltsteuerung

- EVA-Prinzip: Eingabe, Verarbeitung, Ausgabe
- Bisheriger Fokus: RAM (Verarbeitung) und Interrupts (Eingabe)
- Fehlender Teil: Eingriff in die physikalische Welt
- Aktorik: Steuern mechanischer/physikalischer Bauteile (Motoren, Heizungen, Leuchtmittel)
- Ziel: Größen wie Geschwindigkeit oder Helligkeit stufenlos regeln

Notizen

Bei Robotern, 3D-Druckern oder Smart-Home muss der SoC Leistung stufenlos regulieren. Genau hier stoßen wir auf eine harte Grenze digitaler Pins.

EVA — Schema

Lehr-Modell. GPIO steuert, die externe Versorgung leistet.

Sensor, Main, PWM-Timer, Pin-Mux, Treiber, Aktor

Getrennter Leistungsweg, z. B. 12 V, gemeinsame Masse ohne galvanische Trennung

CLINT ist die Systemzeit. Der GPT erzeugt das Pin-Signal und laeuft weiter, wenn Main haengt.

Watchdog und Reset sind die Ueberwachung. Ein Reset trennt die Motorversorgung nicht.

Kein direktes GPIO am Motor.

Abbildung 1: Steuerpfad, Leistungsweg und Watchdog sind getrennte Ebenen.



Notizen

Schematisch: Sensor → SoC → Aktor. Heute steht der Ausgangspfad im Vordergrund.

Das Problem der digitalen Welt

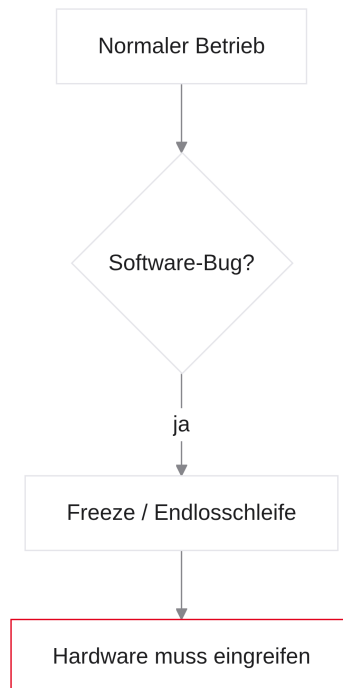
- Die CPU lebt in einer binären Welt
- GPIO-Pin: typisch nur HIGH (z. B. 3,3 V) oder LOW (0 V)
- Dilemma: der Pin kennt nur zwei Pegel. Helligkeit und Kraft folgen nicht linear aus einem Zwischenwert.
- Viele MCU-Pins haben keinen stufenlosen Analogausgang (DAC)
- Lösungsidee: Trägheit der realen Welt ausnutzen

Notizen

Fundamentales Problem digitaler Aktorik: An oder Aus — kein Zwischenwert am Pin. Wir müssen mit Ein/Aus einen Mittelwert simulieren.

Das Problem der Software-Fehler

- Zuverlässigkeit: Was, wenn der Code in `while (1) ;` hängt?
- Folgen: System reagiert nicht; Aktoren können unkontrolliert weiterlaufen
- In sicherheitskritischen Domänen (Medizin, Avionik, Industrie) inakzeptabel
- Bedarf: Hardware, die die CPU überwacht und im Notfall resetten kann



Notizen

Wenn Software tot ist, kann sie keinen Not-Aus auslösen. Der Watchdog ist das zweite große Thema des Blocks — nach der Aktorik.

Agenda für den heutigen Tag

1. Hardware-Counter und Timer-Architektur (warum der CLINT hier nicht reicht)
2. Pulsweitenmodulation (PWM) — Illusion analoger Signale
3. PWM-Erzeugung in Hardware (Compare-Register)
4. System-Sicherheit und Watchdog Timer (WDT)
5. Brückenschlag zum Labor (z. B. Renode und RGB-LEDs)

Notizen

Zuerst Aktorik und Timer-Hardware, dann PWM, danach Watchdog gegen unkontrollierte Aktoren bei Software-Absturz.

Folie

9 / 69

Lernziele Block 4

- Aufbau eines General Purpose Timers (GPT) inkl. Prescaler erklären
- Mathematik der PWM verstehen (Frequenz, Duty Cycle)
- Begründen, warum Software-PWM (Bit-Banging) die CPU belastet und Timing gefährdet
- Watchdog Timer (WDT) korrekt konfigurieren und „füttern“
- Typische Anfängerfehler beim WDT vermeiden

Notizen

Am Ende können Sie Leistungselektronik stufenlos ansteuern, ohne die CPU zu blockieren, und die Idee ausfallsicherer Überwachung mit dem Watchdog einordnen.

Folie

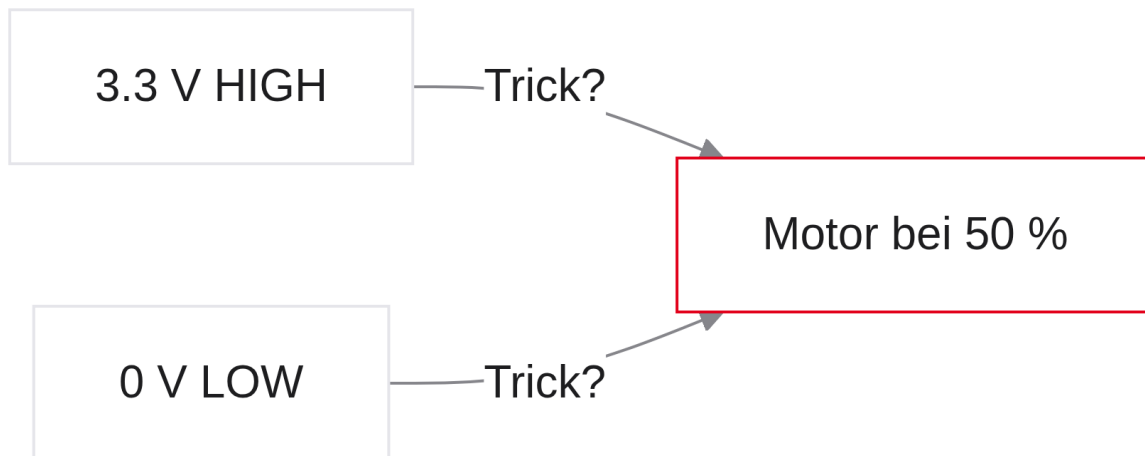
10 / 69

Die Leitfrage des Tages

- SoC-Pin: oft maximal 3,3 V (100 %) oder 0 V (0 %)
- Ziel im Beispiel: 50 Prozent Tastverhältnis. Das ist nicht allgemein halbe Motorleistung.
- Ohne externe Bauteile wie digitale Potentiometer
- Wie dimmt man auf 50 %, wenn der Pin nur „Vollgas“ oder „Aus“ kennt?

Notizen

Wie erzeugen wir „Grau“, wenn wir nur Schwarz und Weiß haben? Die Antwort liegt in schneller Umschaltung — und in Hardware, die das ohne CPU-Last erledigt.

Leitfrage — Schema**Notizen**

Schematisch: zwei Pegel, ein gewünschter Mittelwert — PWM ist der Weg.

CLINT vs. Peripherie-Timer

- Warum nicht einfach `mtime` aus Block 3 für Aktorik?
- CLINT: 64-Bit-System-Timer — läuft durch, darf nicht „wild“ zurückgesetzt werden (OS-Zeitbasis)
- Aktorik braucht Timer, die periodisch bei 0 neu starten und mit Ausgangspins verbunden sind
- Lösung: General Purpose Timers (GPT) in der Peripherie — oft viele pro SoC

Notizen

Die System-Uhr bleibt unangetastet. Für Signale konfigurieren, stoppen und resettet wir GPTs, ohne die OS-Zeit zu verlieren.

Der General Purpose Timer (GPT)

- Flexibles Zählwerk in Hardware, unabhängig vom RISC-V-Kern
- Aufgaben: externe Ereignisse zählen, Zeit stoppen, Pulsmuster auf Pins ausgeben
- Heute: programmierbarer Signalgenerator — nicht „System-Uhr“
- Konfiguration über Memory-Mapped Register (MMIO)

Notizen

In Datenblättern oft TIM1, TIM2, ... — eigene kleine Zähl-Subsysteme mit reicher Konfiguration.

Anatomie eines Hardware-Counters

Drei Kernbausteine:

- Clock Source: oft Systemtakt (z. B. 100 MHz)
- Prescaler (PSC): Vorteiler, drosselt den Takt
- Counter-Register (CNT): Zählwert (typisch 16- oder 32-Bit)

Jeder Tick hinter dem Prescaler erhöht CNT um 1.

Notizen

Ohne Prescaler würde ein 16-Bit-Zähler bei 100 MHz sehr schnell überlaufen — der PSC ist das Getriebe.

Counter-Kette — Schema

Kernbeispiel: $f_{tim} = 100\text{ MHz}$, $PSC = 99$, Teiler 100, $f_{cnt} = 1\text{ MHz}$, 1 us/Tick.

$PSC = 0$ teilt durch 1. $PSC = 3$ ist nur die Detailsicht eines Teilers durch 4.

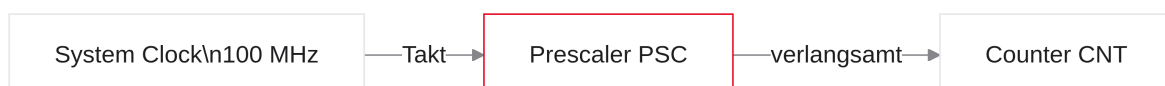
Separates Beispiel: $PSC = 9999$ ergibt 10 kHz und 100 us pro Tick.

16 Bit: Werte 0..65535 sind 65536 Zustände.

Bei 100 MHz ohne Teiler: 0,65536 ms. Bei 1 MHz: 65,536 ms.

Up, Down und Center-Aligned haben eigene Endpunkte. Nicht eine Formel fuer alle.

Abbildung 2: 100 MHz durch PSC+1. PSC 99 ergibt 1 MHz. 16 Bit hat 65536 Zustände.



Notizen

Schematisch: Systemtakt $\rightarrow$ Prescaler $\rightarrow$ Counter.

Folie

16/69

Der Prescaler (Vorteiler)

- Problem: $100 \text{ MHz} = 10^8 \text{ Ticks/s}$; 16-Bit-Max $\approx 65\,535 \rightarrow$ Überlauf in unter 1 ms
- Lösung: Prescaler lässt nur jeden n -ten Takt durch
- Typisch: Registerwert PSC , Teilung durch $PSC + 1$ (weil 0 mitzählt)
- Beispiel: $PSC = 99 \rightarrow$ Teiler 100 $\rightarrow$ langsames Zählen, längere Messfenster

Notizen

$PSC = 0$ heißt nicht „Stopp“, sondern Teiler 1 (volle Geschwindigkeit). Die „+1“ in der Formel ist laborrelevant.

Folie

17/69

Mathematik: Zählfrequenz

$$f_{\text{cnt}} = \frac{f_{\text{clock}}}{PSC + 1}$$

Beispiel: $f_{\text{clock}} = 100 \text{ MHz}$, $PSC = 9999$

$$f_{\text{cnt}} = \frac{10^8}{10^4} = 10 \text{ kHz}$$

$\rightarrow$ Counter erhöht sich 10 000-mal pro Sekunde (alle 0,1 ms).

Notizen

Im Labor müssen Sie f_{cnt} aus Datenblatt-Takt und Prescaler berechnen können — inkl. der +1.

Das Auto-Reload Register (ARR)

- Wo endet das Zählen? → Auto-Reload / Period Register (ARR)
- Absolute Obergrenze des Zählers
- Erreicht CNT den ARR-Wert: Hardware setzt typisch auf 0 zurück
- Dieser Rücksetzzyklus definiert die Periode des Timers
- Achtung Datenblatt: Periode oft ARR oder $ARR + 1$ Ticks

Notizen

ARR ist das Limit. Ob die Periode ARR oder $ARR + 1$ Zählschritte hat, steht im Datenblatt (Hersteller unterscheiden sich) — vor Duty-Cycle-Rechnungen immer nachschlagen.

Zähl-Modus: Up-Counting

- Standard für viele GPT-Anwendungen
- Ablauf: 0 → hoch bis ARR → Reset auf 0
- Signalform: Sägezahn
- Einsatz: Zeitmessung, periodische Interrupts, einfache PWM

Phase	CNT
Start	0
Mitte	steigt
Grenze	ARR
Danach	0 (neu)

Notizen

Up-Counting zeichnet den klassischen Sägezahn — Basis für Edge-Aligned PWM später.

Zähl-Modus: Down-Counting

- Umgekehrt: Start bei ARR, herunter bis 0
- Bei 0 oft Event/Interrupt; Neu laden mit ARR
- Signalform: fallender Sägezahn
- Einsatz: Timeouts; verwandt mit manchen SysTick-Konzepten (z. B. Cortex-M)

Notizen

Vergleich mit 0 kann in Hardware einfach sein — funktional oft austauschbar mit Up-Counting plus ARR.

Zähl-Modus: Center-Aligned

- Fortgeschrittener Modus: 0 → ARR → wieder zurück nach 0 (Richtungsumkehr)
- Signalform: Dreieck
- Einsatz: moderne Motorsteuerung (z. B. FOC), Wechselrichter

Phase	Verhalten
Aufwärts	0 → ARR
Abwärts	ARR → 0
Form	Dreieck

Notizen

Center-Aligned ist besonders relevant für BLDC/FOC — symmetrische PWM und weniger störende Schaltmuster.

Fragen? — Counter und GPT

- Rolle von Prescaler (PSC) und Auto-Reload (ARR) klar?
- Zählfrequenz f_{cnt} berechnen können?
- Up-Counting vs. Center-Aligned unterschieden?
- Warum Datenblatt zu ARR vs. $ARR + 1$ prüfen?

Notizen

PSC drosselt, ARR begrenzt. Als Nächstes: daraus eine „analoge“ Welt — PWM.

Folie

23 / 69

Die analoge Illusion

- Zurück zur Leitfrage: 50 % Motorleistung mit binärem Pin?
- Trick: Pin sehr schnell ein- und ausschalten
- Beispiel: 1 kHz, je Hälfte der Periode HIGH/LOW
- Motor und Auge sind zu träge, um dem Stottern zu folgen
- Effekt: schnelles Schalten. Wahrnehmung und mechanische Wirkung sind davon zu trennen.

Notizen

Wenn Licht sehr schnell blinkt, mittelt das Auge die Helligkeit. Motoren mitteln über Masse und Induktivität ähnlich.

Folie

24 / 69

Was ist PWM?

- Pulsweitenmodulation (PWM, Pulse Width Modulation)
- Wichtige Schnittstelle: digitale Steuerung und Leistungselektronik
- Grundfrequenz bleibt konstant; die Pulsweite (HIGH-Anteil) wird moduliert
- Breiter Puls → viel Energie; schmaler Puls → wenig Energie

Notizen

PWM steuert Lüfter, dimmbare LEDs und Motoren: Raster bleibt, nur die HIGH-Dauer ändert sich.

Periode und Frequenz

- Periode T : Dauer eines Zyklus (HIGH + LOW)
- Frequenz $f = 1/T$
- Zu niedrige f (z. B. 10 Hz): sichtbares Flackern, ruckelige Motoren
- Richtwerte (Faustregeln): LEDs oft $\gtrsim 200$ Hz; Motoren oft 10–20 kHz (weniger hörbares Fiepen)

Abschnitt	Bedeutung
HIGH	Energie ein
LOW	Energie aus
T	ein voller Zyklus

Notizen

Die Frequenz ist das Raster. Für LEDs gegen Flackern, für Motoren oft höher gegen hörbare Töne.

Der Duty Cycle (Tastverhältnis)

- Wichtigste PWM-Größe: Anteil der HIGH-Zeit an der Periode

$$\text{Duty Cycle} = \frac{t_{\text{HIGH}}}{T}$$

- Prozentsatz der Energie, die wir einbringen
- Periode/Frequenz konstant; wir ändern den Duty Cycle

Notizen

Im Code regeln Sie typischerweise den Duty Cycle (Dimmer, Sollgeschwindigkeit) — nicht die Grundfrequenz.

Duty Cycle in Prozent

Bei konstanter Frequenz:

Duty Cycle	Wirkung (qualitativ)
10 %	kurze Pulse; schwach
50 %	halbe aktive Zeit
90 %	fast dauerhaft HIGH

Die Periode bleibt gleich — nur die fallende Flanke wandert.

Notizen

Modulation der Pulsweite bei festem Raster: so steuern Sie die abgegebene Energie.

Mittlere Spannung berechnen

$T = 1 \text{ ms}$, Pegel 0/3,3 V. 25, 50 und 75 Prozent sowie die Konstanten 0 und 100 Prozent.

25 Prozent: Mittelwert 0,825 V, RMS 1,65 V. Die Mittelwertlinie ist keine geglättete Pin-Spannung.

Ideale ohmsche Last, $R = 1 \text{ k}\Omega$: $P = D \cdot V^2 / R = 2,7225 \text{ mW}$.

Konstante 0,825 V an derselben Last: 0,680625 mW. Nicht V_{avg}^2 / R fuer das Rechteck.

Motor: Strom, Gegen-EMK und Treiber. LED: ungefähre Lichtleistung, Wahrnehmung extra.

Multimeter nur mit Messart und Bandbreite deuten. 50 Prozent Duty ist nicht halbe Motorleistung.

Abbildung 3: 25 Prozent: Mittelwert 0,825 V, RMS 1,65 V. Ohmsche Rechteckleistung ist nicht V_{mittel} zum Quadrat durch R .

Zeitlicher Mittelwert über die Periode (nicht der Effektivwert/RMS):

$$\bar{V} = V_{\text{Pin}} \times \text{Duty Cycle}$$

Beispiel: $V_{\text{Pin}} = 3,3 \text{ V}$, Duty Cycle = 25 % = 0,25

$$\bar{V} = 3,3 \text{ V} \times 0,25 = 0,825 \text{ V}$$

Zum Vergleich RMS eines Rechtecksignals: $V_{\text{RMS}} = V_{\text{Pin}} \cdot \sqrt{D}$ — bei 25 % also $\approx 1,65 \text{ V}$. Bei idealer ohmscher Last gilt $P = D \cdot V^2 / R$, nicht V_{avg}^2 / R fuer das ungefilterte Rechteck.

Notizen

Mittelwert, RMS und Leistung sind drei Größen. Ein langsames Multimeter zeigt nicht automatisch den Mittelwert; Modus und Bandbreite entscheiden. 50 Prozent Duty ist keine halbe Motorleistung.

Folie

29 / 69

Auflösung (Resolution)

$f_{\text{cnt}} = 1 \text{ MHz}$, berechnetes Modell, keine Messung.

1 kHz: $N = 1000$, $\text{ARR} = 999$, Schritt 0,1 Prozent

4 kHz: $N = 250$, $\text{ARR} = 249$, Schritt 0,4 Prozent

20 kHz: $N = 50$, $\text{ARR} = 49$, Schritt 2 Prozent

Ein 16-Bit-Zähler liefert bei kurzer Periode nicht 16 Bit Tastverhältnis.

Abbildung 4: Bei 1 MHz: 1 kHz hat 0,1 Prozent Schritt, 20 kHz hat 2 Prozent.

- Feinheit des Dimmens, oft in Bit; gebunden an Counter/ARR
- 8 Bit: 256 Stufen $\Rightarrow$ Schritt $\approx 100\%/256 \approx 0,39\%$
- 16 Bit: 65 536 Stufen — sehr feines Fading
- Zu grobe Stufen: sichtbarer Treppeneffekt bei LEDs

Notizen

ARR und Counterbreite begrenzen die nutzbare Auflösung. Für sanftes LED-Fading oft 12–16 Bit wünschenswert.

Warum nicht Software? (Bit-Banging)

Ideal 50 us / 50 us waere 10 kHz. Laufzeit der Funktionen kommt dazu.

Busy-Wait bindet Main. Freigegebene Interrupts koennen trotzdem laufen.

Konstruiert: fallende Flanke 20 us zu spaet. HIGH 70 us, LOW 50 us, Periode 120 us, etwa 58,3 Prozent.

ISR-PWM gibt Zeit frei, bleibt aber softwarelatenzbehaftet.

Hardware-PWM folgt dem Timer, nicht der ISR-Latenz.

Abbildung 5: 20 us verspaetete Flanke: Periode 120 us, etwa 58,3 Prozent. Hardware-PWM folgt dem Timer.

```
/* Software-PWM - blockiert die CPU */  
while (1) {  
    set_pin_high();  
    delay_us(50);    /* 50 % HIGH */  
    set_pin_low();  
    delay_us(50);  
}
```

- CPU zu 100 % im Busy Waiting — keine andere Arbeit
- Jeder Interrupt zerstört das Timing

Notizen

Bit-Banging ist in Embedded-Systemen meist ungeeignet: die CPU ist gebunden und kann Sensorik/Netzwerk nicht zuverlässig bedienen.

Timing-Problem: Jitter

- Auch mit Timer-ISRs statt `delay()`: andere Interrupts (z. B. UART) verzögern Toggles
- Jitter: unbestimmte Verschiebung der Flanken
- Folge: Duty Cycle schwankt → Motor zuckt, LED blitzt

Notizen

Software-PWM leidet unter Prioritätskonflikten. Für Motorlager und präzise Regelung ist Jitter problematisch.

Hardware-PWM rettet den Tag

- Philosophie: konfigurieren und vergessen
- GPT ist oft direkt mit dem Pin verdrahtet (Alternate Function / Pin-Mux)
- CPU setzt Frequenz und Duty Cycle einmal — dann weiterrechnen oder schlafen
- Hardware erzeugt das Rechteck autark und jitterarm

Notizen

Der GPT toggelt in Silizium — ohne Taktzyklen der RISC-V-Pipeline für jedes Flip.

Praxis: RGB-LEDs

- RGB-LED: drei Dioden (Rot, Grün, Blau)
- Je Farbe ein PWM-Kanal
- Beispiel: R 100 %, G 50 %, B 0 % → wahrgenommenes Orange
- Laborziel (geplant): Farbverläufe über Duty Cycles

Notizen

Klassischer Labor-Use-Case: drei Kanäle, Mischen beliebiger RGB-Farben über Tastverhältnisse.

Nichtlinearität des Auges

- Duty Cycle 50 % wirkt oft nicht „halb so hell“
- Helligkeitsempfindung näherungsweise logarithmisch
- Bei 50 % PWM oft schon subjektiv sehr hell
- Gegenmaßnahme: Gamma-Korrektur (Potenz/LUT), damit lineares Fading natürlich wirkt

Notizen

Im Labor: lineares Hochdimmen wirkt am Anfang schnell hell, danach kaum noch Zuwachs — deshalb oft exponentielle Duty-Cycle-Kurven.

Fragen? — PWM-Grundlagen

- Periode T und Duty Cycle klar?
- Warum ist Bit-Banging riskant für Stabilität und Timing?
- V_{eff} nachvollziehbar?

Notizen

Als Nächstes: Counter + Compare-Register — wie die Hardware das Signal erzeugt.

Counter trifft Compare-Register

- Bekannt: CNT zählt, ARR begrenzt die Periode
- Neu: Capture/Compare Register (CCR) — Schwelle für den Duty Cycle
- Merksatz:
 - ARR → Frequenz / Periode
 - CCR → Duty Cycle

Notizen

In C programmieren Sie typischerweise ARR (Raster) und CCR (Schieberegler für die Pulsweite).

Der Komparator im Timer

- Hardware vergleicht fortlaufend CNT und CCR
- Typische Logik (vereinfacht): solange $\text{CNT} < \text{CCR} \rightarrow \text{Pin HIGH}$, sonst LOW
- Läuft automatisch — ohne Software-Latenz pro Takt

Notizen

CCR verkleinern → kürzere HIGH-Phase → dunklere LED / weniger Leistung.

Edge-Aligned PWM

Up-Counting 0..999, ARR = 999, CCR = 250, Active-High wenn CNT kleiner CCR.

Ticks 0..249 aktiv: 250 Intervalle, Ticks 250..999 inaktiv: 750 Intervalle.

$N = 1000$, $f_{\text{PWM}} = 1 \text{ kHz}$, $t_{\text{HIGH}} = 250 \text{ us}$, $D = 25 \text{ Prozent}$.

CCR 0, 250, 500, 1000 ergeben 0, 25, 50 und 100 Prozent.

Match ist CNT gleich CCR. Danach ist der Ausgang inaktiv, weil CNT nicht kleiner CCR ist.

Der Pin braucht keinen Compare-Interrupt. ARR = 999 ist ein voller Zustand.

Abbildung 6: CNT 0 bis 999, CCR 250: 250 aktive und 750 inaktive Tickintervalle.

- Up-Counting (Sägezahn) + Vergleich mit CCR → Edge-Aligned PWM
- Beispiel: $ARR = 999$, $CCR = 250$ → exakt 25 % Duty Cycle (Periode 0 ... 999 = 1000 Takte)
- Pin HIGH von 0 bis unter CCR, danach LOW bis ARR, dann Reset

Notizen

Flankenausgerichtet: ein Sägezahn-Durchlauf, eine klare HIGH-Phase am Anfang der Periode. Mit $ARR = 999$ und Zählung 0 ... 999 ist die Periodenlänge exakt 1000 — analog zur „PSC+1“-Falle beim Prescaler. Ob Ihr Chip ARR oder $ARR + 1$ als Periodenlänge zählt, steht im Datenblatt.

Schritt 1: Start — Pin HIGH

- Start: $CNT = 0$
- Vergleich: $0 < 250$ ($CNT < CCR$) → ja → Pin HIGH (3,3 V)
- Timer zählt weiter; die CPU kann andere Aufgaben erledigen
- LED leuchtet in der HIGH-Phase

CNT	CCR	Pin
0 ... 249	250	HIGH
≥ 250	250	LOW (nächster Schritt)

Notizen

Zyklusbeginn: Counter unter CCR → Hardware hält den Pin HIGH. Die CPU ist nicht am Toggeln beteiligt.

Schritt 2: Counter erreicht CCR (Match)

- Counter erreicht exakt $CCR = 250$
- Vergleich: $250 < 250?$ → nein → Compare Match
- Hardware zieht den Pin sofort auf LOW (0 V)
- Pin bleibt LOW, während CNT weiter bis ARR steigt (250 ... 999)
- LED in dieser Phase aus

Phase	Bedingung	Pin
HIGH	$CNT < CCR$	HIGH
Match	$CNT \geq CCR$	LOW

Notizen

Ab dem Schwellenwert liefert die Hardware keine Energie mehr bis zum Periodenende. Bei $ARR = 999$ und $CCR = 250$ sind das exakt 250 HIGH- und 750 LOW-Takte = 25 %. Prüfen Sie im Datenblatt, ob die Periode ARR oder $ARR + 1$ beträgt.

Schritt 3: Counter erreicht ARR (Neustart)

- CNT erreicht $ARR = 999$
- Auto-Reload: Hardware setzt CNT auf 0
- Vergleich erneut: $0 < 250$ → ja → Pin wieder HIGH
- Ergebnis: Rechteck mit 25 % Duty Cycle — ohne laufende CPU-Arbeit

Notizen

Der Kreis schließt sich: ARR startet den Zyklus neu. Bei z. B. 10 kHz Periodenrate dimmt die Hardware flackerfrei, während die CPU schlafen oder rechnen kann.

Die Polarität (Polarity)

Gewähltes Timer-Modell, kein universelles Registerrezept.

Ungepuffert: CCR von 250 auf 750 bei CNT = 500 kann den schon inaktiven Pin wieder einschalten.

Gepuffert: der neue Wert wird erst beim Update-Ereignis aktiv.

Init: Takt, Pin-Mux, Modus, Polarität, PSC/ARR/CCR, Preload, Flags, Kanal und Timer.

PSC, ARR und CCR allein schalten noch keinen Ausgang.

Abbildung 7: Preload wird erst beim Update aktiv. Drei Kanäle und Gamma-CCR 0, 47, 218, 531, 1000.

Ein ARR, drei CCR. Active-Low: aktiver LED-Anteil ist nicht der HIGH-Anteil.

Common-Cathode und Common-Anode brauchen je Zweig eine Strombegrenzung.

Gamma-Lehrannahme $D = b^2/2$. Bei $N = 1000$: $b = 0 / 0,25 / 0,5 / 0,75 / 1$

ergibt gerundet CCR 0, 47, 218, 531 und 1000.

$R=100$, $G=50$, $B=0$ kann orange wirken. Spektrum und Abgleich sind nicht garantiert.

Abbildung 8: RGB und Gamma sind ein Lehrmodell, keine Farbgarantie.

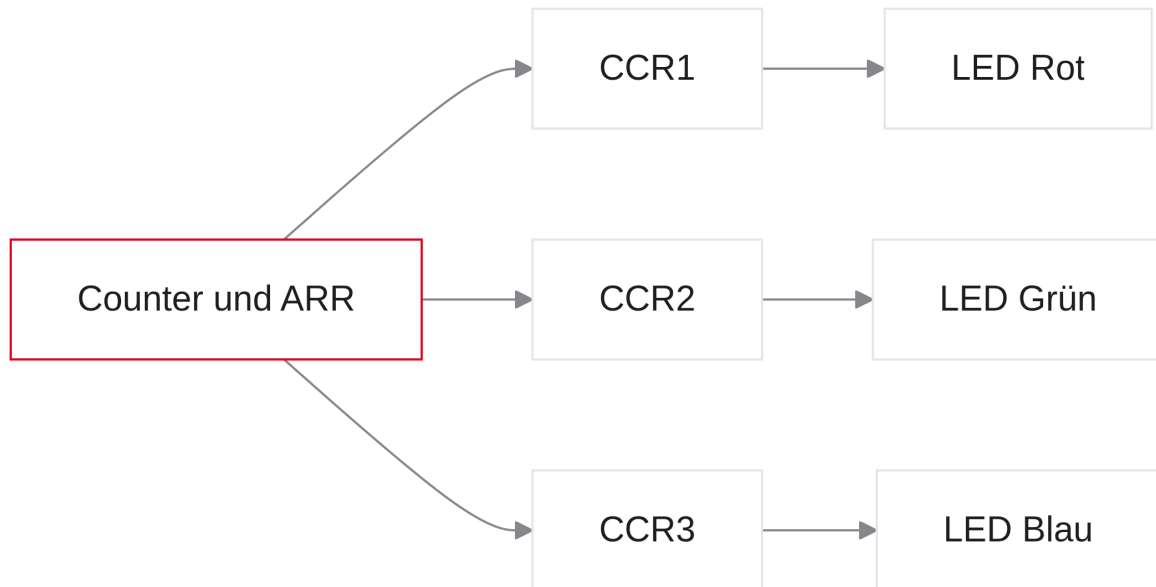
- Bisher Active-High: $CNT < CCR \rightarrow \text{Pin HIGH}$
- Alternative: Active-Low — Logik invertiert
- Wichtig bei invertierter Beschaltung (z. B. Common-Anode-LEDs, manche Treiber)
- Typisch: Polarity-Bit im Timer-Konfigurationsregister

Notizen

Laborfalle: Duty Cycle erhöhen und die LED wird dunkler $\rightarrow$ oft Active-Low. Polarity-Bit setzen, statt die Software-Logik zu verdrehen.

Mehrere Kanäle an einem Timer

- Ein Timer: ein Counter und ein ARR → gemeinsame Frequenz
- Oft 4 Compare-Register (CCR1–CCR4) → bis zu 4 Pins
- Unterschiedliche Duty Cycles möglich, gleiche Grundfrequenz
- Ideal für RGB: ein Rhythmus, drei Helligkeiten



Notizen

Ein GPT speist mehrere Komparatoren: vier PWM-Signale zum „Preis“ eines Timer-Moduls — Frequenz gemeinsam, Duty Cycles unabhängig.

Center-Aligned PWM

- Dreieckszähler: Komparator schaltet beim Hoch- und Herunterzählen
- PWM-Pulse symmetrisch zur Periodenmitte
- Nutzen: Wechselrichter / Motorantrieb — oft weniger störende Harmonische
- Für LEDs reicht meist Edge-Aligned

Notizen

Ausblick Antriebstechnik: zentrierte Pulse können Netzstörungen und Verluste reduzieren. Für Dimmen von LEDs ist Edge-Aligned typisch ausreichend.

Glättung von PWM-Signalen

- Manche Lasten brauchen echte Gleichspannung (z. B. Audio)
- RC-Glied (Widerstand + Kondensator) als Tiefpass am PWM-Pin
- Hohe Frequenzanteile (Kanten) werden gedämpft → Mittelwert bleibt
- Ergebnis: steuerbare Analogspannung — PWM als einfacher DAC

Notizen

Ohne integrierten DAC: PWM + RC-Filter ergibt eine variable Gleichspannung für Zeigerinstrumente oder einfache Analogpfade.

Anwendung: Servomotoren

Drei Anwendungen, drei Perioden. RC ist kein Motortreiber.

RC: 1 kOhm, 10 uF, 1 kHz, 25 Prozent. Tau 10 ms, fc etwa 15,9 Hz, Mittel 0,825 V.

Unbelastete Welligkeit etwa 61,9 mV. 95 Prozent eines Sprungs etwa 30 ms.

Servo: 20 ms, Pulse 1,0 / 1,5 / 2,0 ms. ARR = 19999, CCR = 1000 / 1500 / 2000 bei 1 MHz.

H-Brücke: 3,3 V steuern, z. B. 12 V leisten. Keine komplementären Roh-Gates ohne Totzeit.

Winkel und Wiederholrate sind servospezifisch.

Abbildung 9: RC-Glied, Servo mit 20 ms und H-Brücke sind drei verschiedene Anwendungen.

- Hier ist PWM oft Datenprotokoll, nicht reine Leistungsübertragung
- Typisch: 50 Hz (Periode 20 ms)
- Pulsbreite steuert den Winkel:
 - 1,0 ms → eine Endlage
 - 1,5 ms → Mitte
 - 2,0 ms → andere Endlage
- Motorleistung kommt über separate Versorgung

Notizen

LED-Dimmen: Energieanteil. RC-Servo: Impulsbreite als Information. Die interne Elektronik des Servos setzt den Winkel um.

Anwendung: DC-Motoren und H-Brücke

- Motor nie direkt an GPIO (Strombegrenzung, Chip-Schutz)
- Motortreiber / H-Brücke dazwischen
- SoC: schwaches PWM-Kommando (z. B. 3,3 V)
- H-Brücke: schaltet externe Versorgung (z. B. 12 V) im PWM-Takt

Notizen

MCU = Gehirn, Treiber = Muskel. Der Chip steuert Transistoren/H-Brücke; der Hochstrom fließt nicht durch den GPIO.

Zusammenfassung: Hardware-PWM

- GPT erzeugt Signale autark (kein Bit-Banging)
- Frequenz: Prescaler (PSC) und Auto-Reload (ARR)
- Duty Cycle: Compare-Register (CCR)
- Active-High (typisch): $CNT < CCR \rightarrow \text{Pin HIGH}$
- LEDs direkt (strombegrenzt); Motoren über Treiber/H-Brücke — jitterarm

Notizen

Damit endet der Aktorik-Teil: wenige Register setzen, Hardware erledigt das Rechteck. Als Nächstes: Was, wenn Software einfriert und der Aktor auf hoher Leistung stehen bleibt?

Fragen? — Hardware-PWM

- Zusammenspiel von Counter, ARR und CCR?
- Warum gleiche Frequenz, aber unterschiedliche Duty Cycles auf einem Timer?
- PWM zum Dimmen vs. PWM als Servo-Protokoll?

Notizen

Als Nächstes: Software-Abstürze in kritischen Systemen und der Watchdog.

Szenario: System ohne Reset-Zugang

- Beispiel: Gerät ohne physischen Zugang (Satellit, Implantat, Feldgerät)
- Software hängt in einer Endlosschleife (Freeze)
- Kommunikation und Steuerung fallen aus
- Niemand kann den Reset-Knopf drücken
- Folge: Mission / Funktion dauerhaft verloren — ohne Selbstheilung

Notizen

Auf dem Schreibtisch reicht der Reset-Taster. In autonomen oder unzugänglichen Systemen muss Hardware von Software-Ausfällen ausgehen und selbst eingreifen können.

Warum stürzt Software ab?

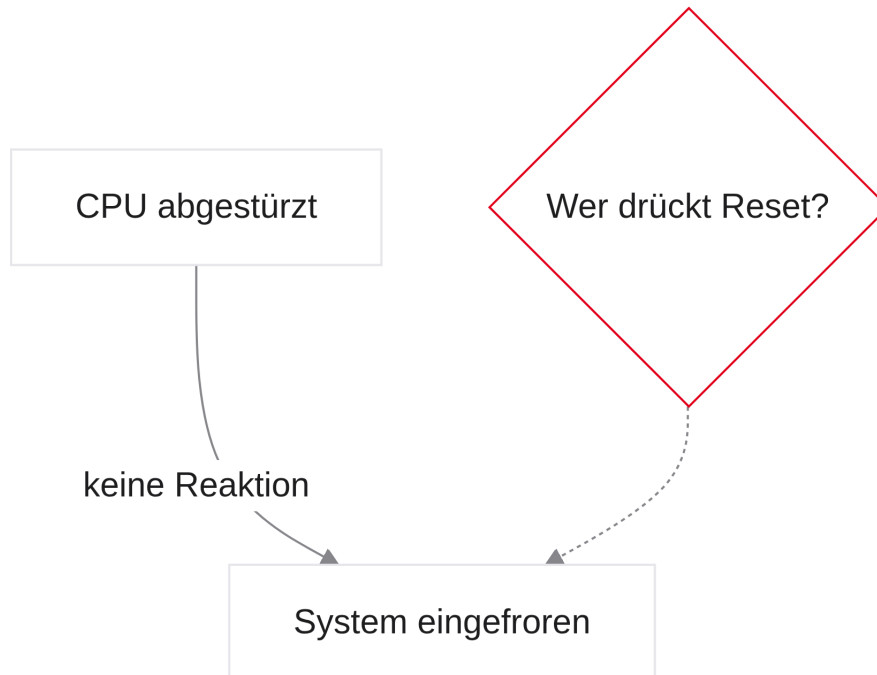
- Endlosschleifen (z. B. Timeout vergessen: `while (sensor_busy) ;`)
- Deadlocks (Multicore, später Block 9)
- Stack Overflow (z. B. zu tiefes Nesting)
- Soft-Errors: Bit-Flips (z. B. Strahlung) → Program Counter / Zustand zerstört

Notizen

Selbst gut getesteter Code kann scheitern. Wir brauchen eine Hardware-Lebensversicherung unabhängig von der CPU-Logik.

Leitfrage 2: Wer resettet die tote CPU?

- OS/C-Code abgestürzt — CPU reagiert nicht mehr
- Wie resettet man, wenn das „Gehirn“ den Reset nicht mehr auslösen kann?
- Antwort: Software nicht vertrauen — unabhängige, einfache Hardware



Notizen

Ein totes Gehirn startet sich nicht selbst. Es braucht eine externe Instanz — den Watchdog.

Der Watchdog Timer (WDT)

- WDT: einfaches Hardware-Modul, getrennt von der ALU
- Ultimatum: z. B. „Reset in 5 s, wenn kein Lebenszeichen“
- Gesunde Software setzt den Zähler periodisch zurück
- Bei Ausfall: Timeout → Hardware zieht die Reset-Leitung

Notizen

Der Watchdog ist eine Zeitbombe, die Software regelmäßig entschärfen muss. Sonst erzwingt er einen System-Reset.

Architektur des WDT

Ideales Timeout 2 s. Feeds bei 0, 0,5 und 1,0 s. Kein weiterer Feed, Reset bei 3,0 s.

Eigener Takt und Zeitfenster sind zwei Eigenschaften.

Ein Fenster-Watchdog muss keinen eigenen Takt haben. Ein unabhängiger WDT kann ein Fenster haben.

0xAA55 ist ein fiktiver Feed. Laborcode braucht die dokumentierte Sequenz.

Nicht jeder Watchdog hat nur Reset. Vorwarnung ist möglich.

Abbildung 10: Feeds bei 0, 0,5 und 1 s, idealer Reset bei 3 s. 0xAA55 ist fiktiv.

- Oft Down-Counter
- Beim Start: Timeout-Wert laden (z. B. äquivalent zu 5 s) und starten
- Zählt rückwärts; bei vielen SoCs nach Aktivierung nicht mehr abschaltbar
- Einfachheit ist Sicherheitsfeature

Notizen

Wenige Features, harte Semantik: einmal scharf, oft nicht per Software zu stoppen — auch gegen Missbrauch nach Kompromittierung.

Der Biss des Hundes

- Counter erreicht 0
- Kein „freundlicher“ Interrupt (CPU könnte taub sein)
- Direkter Eingriff in Reset-/Power-Management
- Chip wie nach Hardware-Reset: CPU, Zustand, Peripherie neu
- Boot ab Reset-Vektor / Flash-Startadresse

Notizen

Der WDT löst einen harten Reset aus — die Freeze-Schleife ist weg, das System startet sauber neu.

Den Hund streicheln (Feed)

- Gesunde Software muss den WDT periodisch zurücksetzen
- „Petting / kicking the dog“: magischen Wert ins Feed-Register schreiben
- Counter springt auf den Startwert zurück

```
void wdt_feed(void) {  
    *(volatile uint32_t *)WDT_FEED_REG = 0xAA55;  
}
```

Notizen

Regelmäßiges Schreiben ins Feed-Register ist das Lebenszeichen: „Ich lebe noch.“

Wo streichelt man den Hund?

- Falsch: Füttern nur in der SysTick-ISR
 - main kann tot sein, Interrupts laufen weiter → kein Reset
- Richtig: in der untersten while (1)-Ebene der Geschäftslogik

```
int main(void) {  
    init_wdt(); /* z. B. 5 s Timeout */  
    while (1) {  
        lese_sensoren();  
        berechne_werte();  
        wdt_feed(); /* nur wenn wir hier ankommen */  
    }  
}
```

Notizen

Klassischer Fehler: Feed in der 1-ms-ISR. Die Hauptschleife hängt, der Tick füttert weiter — der Watchdog „merkt“ den Freeze nicht.

Unabhängige Taktquelle (LSI)

- Was, wenn der Hauptquarz ausfällt?
- Gemeinsamer Takt: CPU und WDT würden gemeinsam einfrieren
- Echter WDT: eigener Low-Speed-Oszillator (LSI, oft ≈ 32 kHz)
- Isolation: Überwachung unabhängig vom CPU-Takt

Notizen

Sicherheit braucht Isolation. Stirbt der Systemtakt, muss der WDT weiterzählen und resetten können.

Window Watchdog (WWDG)

Haengt Main und der Tick fuertert weiter, gibt es keinen Reset.

Fehlt der Fortschritt, bleibt der Feed aus und das Timeout kommt.

Beispiel-Fenster 0,5 bis 2 s: 0,2 s zu frueh, 1 s gueltig, nach 2 s zu spaet.

Raender sind datenblattabhaengig, 10 ms Fading-Feed passt nicht automatisch ins Fenster.

Ein Feed beweist keine korrekte Rechnung.

Abbildung 11: Weiterfuettern aus dem Tick verdeckt einen Haenger. Fenster und Normal-Feed sind verschieden.

- Normaler WDT: erkennt zu langsames Fuettern (Timeout)
- Problem: fehlerhafter Code fuettert zu oft / zu frueh
- Window Watchdog: Fuettern nur in einem Zeitfenster erlaubt
- Zu frueh oder zu spaet → Reset

Zeitpunkt	Bewertung
vor Fenster	Reset (zu frueh)
im Fenster	OK
nach Timeout	Reset (zu spaet)

Notizen

Für kritische Systeme: das Fenster beweist, dass die Software im erwarteten Takt läuft — nicht nur „irgendwie noch fuettert“.

Zusammenfassung Watchdog

- WDT: Netz bei Freeze, Deadlocks, entgleistem Ablauf
- Autarker Down-Counter → harter System-Reset bei 0
- Software setzt periodisch zurück (Feed)
- Feed in der Main-Loop, nicht in der Timer-ISR
- Window Watchdog: auch zu schnelles Abarbeiten erkennen

Notizen

Mit korrekt platziertem Feed überlebt das System viele Software-Ausfälle durch Neustart. Im Labor (geplant) lässt sich der Freeze bewusst provozieren.

Fragen? — Watchdog

- Warum eigener Takt (nicht CPU-Takt)?
- Warum ist Feed in der Timer-ISR ein Designfehler?
- Welches Problem löst der Window Watchdog zusätzlich?

Notizen

Als Nächstes: PWM und Watchdog zusammen — Ausblick auf das Labor in Renode.

Ausblick Labor: smarte Lampe

Geplantes Szenario:

- Timer als PWM-Generator; virtuelle RGB-LEDs in Renode
- Fading in der Main-Schleife
- Watchdog aktivieren
- Virtueller Taster → künstliche `while (1) ;` (Freeze provozieren)

Notizen

Details folgen im Übungsblatt. Ziel: PWM-Fading und Watchdog-Reset im Simulator nachvollziehen.

MMIO-Register für den Timer

Beispielhafte Adressen (fiktiv — immer Datenblatt prüfen):

```
#define TIMER_BASE 0x40000000
#define TIMER_PSC  (*(volatile uint32_t *) (TIMER_BASE + 0x00))
#define TIMER_ARR  (*(volatile uint32_t *) (TIMER_BASE + 0x04))
#define TIMER_CCR1 (*(volatile uint32_t *) (TIMER_BASE + 0x0C))

/* Registerausschnitt, fiktiv. Kein vollstaendiger Treiber. */
void init_pwm_registers(void) {
    TIMER_PSC = 99;      /* Teiler 100 */
    TIMER_ARR = 999;     /* N = 1000 */
    TIMER_CCR1 = 0;
}
/* Danach noch noetig: Takt, Pin-Mux, Modus, Polaritaet,
   Preload, Kanal- und Timerfreigabe laut Datenblatt. */
```

- PSC: Takt drosseln
- ARR: Periode / Frequenz
- CCR1: Duty Cycle Kanal 1

Notizen

Hardware-nahe C-Programmierung: Werte aus dem Datenblatt in MMIO-Register schreiben.

Hilfsfunktion: Prozent → CCR

```
void set_led_brightness(uint8_t percent) {
    if (percent > 100) percent = 100;
    const uint32_t N = 1000u; /* ARR + 1 im Kernmodell */
    uint32_t ccr = ((uint32_t)percent * N) / 100u;
    TIMER_CCR1 = ccr;
}
```

- 0, 25, 50 und 100 Prozent ergeben CCR 0, 250, 500 und 1000.
- Kleine Abstraktion erleichtert die Main-Logik

Notizen

Typische HAL-Schicht: `set_led_brightness(50)` statt roher Registerarithmetik in jedem Aufrufer.

Folie

65/69

Fading in der Main-Schleife

```
int main(void) {
    init_pwm();
    int brightness = 0;
    while (1) {
        set_led_brightness(brightness);
        brightness = (brightness + 1) % 101; /* Sprung 100 auf 0
        /* Deadline aus Block 3, kein blockierendes Warten als G
        wdt_feed();
    }
}
```

- Die Rampe springt von 100 auf 0. Ein Feed beweist nur, dass diese Schleife noch laeuft.

Notizen

Sauberer: Wartezeiten über SysTick statt `busy delay`. Wichtig bleibt: Feed nur hier, nicht in der ISR.

Den Crash provozieren

Letzter Feed 1,0 s, Main steht ab 1,2 s, idealer Reset 3,0 s.

1,8 s läuft die PWM ohne frischen Main-Fortschritt weiter.

Erkennung, Ausgang aus, Neustart und sicherer Aktorzustand sind verschiedene Zeiten.

Lampe: aus. Motor: aus, bremsen oder halten sind nicht dasselbe.

Enable-Pin während Reset definiert halten. Externe Versorgung bleibt.

Nominell 2 s bei +/-20 Prozent Takt: etwa 1,667 bis 2,5 s. Keine Auslegungsempfehlung.

Abbildung 12: Von 1,2 bis 3,0 s läuft PWM ohne Main. Reset und sicherer Aktorzustand fallen auseinander.

- Watchdog-Timeout z. B. 2 s
- Button → künstlicher Deadlock ohne Feed

```
if (button_pressed()) {
    printf("Freeze - warte auf WDT...\n");
    while (1) {
        /* kein wdt_feed() */
    }
}
```

Notizen

PWM-Hardware kann die LED-Helligkeit halten, während `main` hängt. Nach Timeout: harter Reset durch den WDT.

Reset-Ursache lesen

- Nach Reboot: warum neu gestartet?
- Reset-Reason-Register (Power-/Reset-Block)
- Beim Boot zuerst auswerten

```
void check_reset_reason(void) {
    uint32_t reason = *(volatile uint32_t *)RESET_REASON_REG;
    if (reason & WDT_RESET_FLAG) {
        printf("WDT-Reset nach Freeze\n");
        /* optional: loggen / sicherer Modus */
    }
}
```


Notizen

Professionelle Systeme unterscheiden Power-On, Pin-Reset und WDT-Reset — und reagieren darauf (Log, Warnung, degradierter Betrieb).

Folie

68 / 69

Best Practices: WDT-Intervall

- Zu kurz: Fehlalarme bei kurzer CPU-Last (z. B. schwere Berechnung)
- Zu lang: sichtbarer Freeze; Aktoren wirken zu lange unkontrolliert
- Praxis oft: etwa 0,5–3 s, abhängig von Mechanik und Risiko

Notizen

Das Intervall ist eine Designentscheidung: sicherheitskritische Aktoren brauchen kürzere Timeouts als unkritische Anzeigen.

Folie

69 / 69

Zusammenfassung Block 4

- Aktorik über PWM: Illusion steuerbarer Mittelwerte
- GPT: ARR (Frequenz), CCR (Duty Cycle) — autark im Hintergrund
- Kein Bit-Banging: weniger CPU-Last und Jitter
- WDT: harter Reset bei Software-Freeze
- Feed nur in der Main-Loop, nie allein in der Timer-ISR

Vielen Dank — wir sehen uns im Labor.

Notizen

Block 4 schließt EVA und funktionale Grundsicherheit. Im Renode-Labor setzen Sie PWM und Watchdog praktisch um.